DCGAN on Fashion-MNIST (modified from GAN_CIFAR10)

Name: Snehal Pagare | **PRN:** 202502110003

Aim: Train a Deep Convolutional GAN (DCGAN) on the Fashion-MNIST clothing-image dataset, generate synthetic clothing images, and evaluate it using the Generator/Discriminator loss curves and the generated images.

Run in Google Colab: `Runtime > Change runtime type > GPU`, then `Runtime > Run all`.

1. Load and view the Fashion-MNIST dataset

Fashion-MNIST has 60,000 training images of clothing items in 10 classes (T-shirt, trouser, pullover, dress, coat, sandal, shirt, sneaker, bag, ankle boot). Each image is **28 x 28 grayscale** (1 channel), so the model input shape is `(28, 28, 1)`.

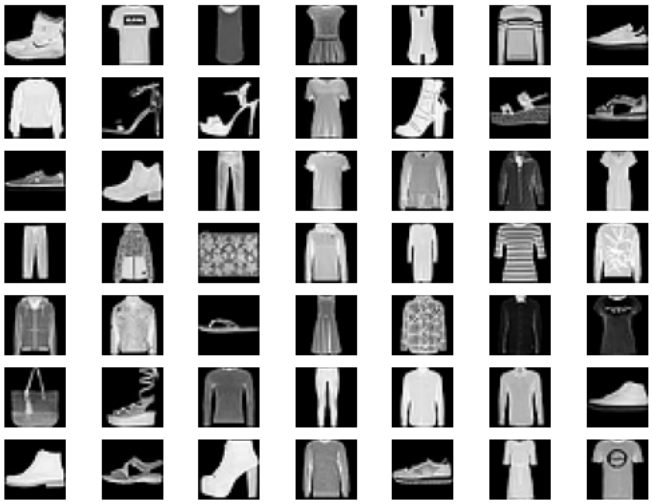
```
# Snehal Pagare | PRN: 202502110003
# example of loading and plotting the Fashion-MNIST dataset
from tensorflow.keras.datasets import fashion_mnist
load_data = fashion_mnist.load_data   # used later by load_real_samples()
from matplotlib import pyplot

# load the images into memory
(trainX, trainy), (testX, testy) = load_data()

# plot images from the training dataset
for i in range(49):
    # define subplot
    pyplot.subplot(7, 7, 1 + i)
    # turn off axis
    pyplot.axis('off')
    # plot raw pixel data (grayscale)
    pyplot.imshow(trainX[i], cmap='gray')

pyplot.show()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```



```
# shape is an attribute, not a function (fixed: trainX.shape, not trainX.shape())
print(trainX.shape)   # (60000, 28, 28) full Fashion-MNIST; load_real_samples() keeps only a 6000-image subset
```

```
(60000, 28, 28)
```

2. Discriminator

A CNN classifier that outputs the probability that an image is **real** (1) or **fake** (0). Strided convolutions downsample 28x28 -> 14x14 -> 7x7.

```
# example of defining the discriminator model
from tensorflow.keras.models import Sequential
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.layers import Input
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Conv2D
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import LeakyReLU
from tensorflow.keras.utils import plot_model
import numpy as np

# define the standalone discriminator model
def define_discriminator(in_shape=(28,28,1)):
    model = Sequential()
    model.add(Input(shape=in_shape))

    # downsample 28x28 -> 14x14
    model.add(Conv2D(64, (3,3), strides=(2,2), padding='same'))
    model.add(LeakyReLU(0.2))
    model.add(Dropout(0.4))

    # downsample 14x14 -> 7x7
    model.add(Conv2D(64, (3,3), strides=(2,2), padding='same'))
    model.add(LeakyReLU(0.2))
    model.add(Dropout(0.4))

    # classifier
    model.add(Flatten())
    model.add(Dense(1, activation='sigmoid'))

    # compile model: sigmoid output (probability) + binary cross-entropy loss are a matching pair
    opt = Adam(learning_rate=0.0002, beta_1=0.5)
    model.compile(loss='binary_crossentropy', optimizer=opt, metrics=['accuracy'])
    return model
```

```
# define model
model = define_discriminator()

# summarize the model
model.summary()

# plot the model (needs pydot/graphviz, available in Colab; skipped if missing)
try:
    plot_model(model, to_file='discriminator_plot.png', show_shapes=True, show_layer_names=True)
except Exception as e:
    print('plot_model skipped:', e)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 14, 14, 64)	640
leaky_re_lu (LeakyReLU)	(None, 14, 14, 64)	0
dropout (Dropout)	(None, 14, 14, 64)	0
conv2d_1 (Conv2D)	(None, 7, 7, 64)	36,928
leaky_re_lu_1 (LeakyReLU)	(None, 7, 7, 64)	0
dropout_1 (Dropout)	(None, 7, 7, 64)	0
flatten (Flatten)	(None, 3136)	0
dense (Dense)	(None, 1)	3,137

Total params: 40,705 (159.00 KB)
Trainable params: 40,705 (159.00 KB)

3. Real samples

`load_real_samples()` loads Fashion-MNIST, keeps a **small subset of 6,000 images** (instead of all 60,000, so training is quick), adds the channel dimension `(N, 28, 28, 1)` and scales pixels from [0, 255] to [-1, 1] so they match the Generator's `tanh` output range.

```
# Funtion to load the dataset and scale it
# load and prepare Fashion-MNIST training images
def load_real_samples(n_images=6000):
    # load Fashion-MNIST dataset
```

```
    (trainX, _), (_, _) = load_data()
    # use only a small subset (first 6000 of the 60000 images) to keep training short and easy
    trainX = trainX[:n_images]
    # add the channel dimension: (60000, 28, 28) -> (60000, 28, 28, 1)
    X = np.expand_dims(trainX, axis=-1)
    # convert from unsigned ints to floats
    X = X.astype('float32')
    # scale from [0, 255] to [-1, 1]
    X = (X - 127.5) / 127.5
    return X
```

```
X = load_real_samples()
print(X.shape, X.min(), X.max())   # expect (6000, 28, 28, 1) -1.0 1.0
```

```
(6000, 28, 28, 1) -1.0 1.0
```

```
# select real samples   128 samples it will train in batches 64- real, 64-fake   y is label as 1 because it is real
def generate_real_samples(dataset, n_samples):
    # choose random instances
    ix = np.random.randint(0, dataset.shape[0], n_samples)
    # retrieve selected images
    X = dataset[ix]
    # generate 'real' class labels (1)
    y = np.ones((n_samples, 1))
    return X, y
```

4. Fake samples and a quick discriminator test

This first `generate_fake_samples` makes **random-noise** images (no generator yet). It is only used to test the discriminator, and it is redefined later (section 5) to use the Generator.

```
# generate n fake samples with class labels (random noise images, for testing the discriminator only)
def generate_fake_samples(n_samples):
    # generate uniform random numbers in [0,1]
    X = np.random.rand(28 * 28 * 1 * n_samples)
    # update to have the range [-1, 1]
    X = -1 + X * 2
    # reshape into a batch of grayscale images
    X = X.reshape((n_samples, 28, 28, 1))
    # generate 'fake' class labels (0)
    y = np.zeros((n_samples, 1))
    return X, y
```

```
# train the discriminator model temporarily
def train_discriminator(model, dataset, n_iter=20, n_batch=128):
    half_batch = int(n_batch / 2)
    # manually enumerate epochs
    for i in range(n_iter):
        # get randomly selected 'real' samples
        X_real, y_real = generate_real_samples(dataset, half_batch)
        # update discriminator on real samples
        _, real_acc = model.train_on_batch(X_real, y_real)
        # generate 'fake' examples
        X_fake, y_fake = generate_fake_samples(half_batch)
        # update discriminator on fake samples
        _, fake_acc = model.train_on_batch(X_fake, y_fake)
        # summarize performance
        print('>%d real=%.0f%% fake=%.0f%%' % (i+1, real_acc*100, fake_acc*100))

# Example Training
model = define_discriminator()
dataset = load_real_samples()
train_discriminator(model, dataset)
```

```
>1 real=41% fake=46%
>2 real=53% fake=54%
>3 real=57% fake=59%
>4 real=61% fake=62%
>5 real=64% fake=65%
>6 real=66% fake=66%
>7 real=68% fake=69%
>8 real=71% fake=71%
>9 real=73% fake=74%
>10 real=74% fake=75%
>11 real=76% fake=77%
>12 real=78% fake=79%
>13 real=79% fake=80%
>14 real=80% fake=81%
>15 real=81% fake=82%
```

```
>16 real=82% fake=83%
>17 real=83% fake=84%
>18 real=84% fake=85%
>19 real=85% fake=85%
>20 real=86% fake=86%
```

5. Generator

Turns a 100-dimensional random vector (latent point) into a 28x28x1 image: a Dense layer makes a 7x7x128 feature map, two `Conv2DTranspose` layers upsample 7 -> 14 -> 28, and a final `Conv2D` with **tanh** outputs values in [-1, 1] (same range as the real images).

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense, Reshape, Conv2D, Conv2DTranspose, LeakyReLU
from tensorflow.keras.utils import plot_model

# define the standalone generator model
def define_generator(latent_dim):
    model = Sequential()
    model.add(Input(shape=(latent_dim,)))

    # foundation for 7x7 image
    n_nodes = 128 * 7 * 7
    model.add(Dense(n_nodes))
    model.add(LeakyReLU(0.2))
    model.add(Reshape((7, 7, 128)))

    # upsample to 14x14
    model.add(Conv2DTranspose(128, (4,4), strides=(2,2), padding='same'))
    model.add(LeakyReLU(0.2))

    # upsample to 28x28
    model.add(Conv2DTranspose(128, (4,4), strides=(2,2), padding='same'))
    model.add(LeakyReLU(0.2))

    # output layer: 1 channel (grayscale), tanh -> [-1, 1]
    model.add(Conv2D(1, (7,7), activation='tanh', padding='same'))
    return model

# define size of latent space & model summary
latent_dim = 100
model = define_generator(latent_dim)
model.summary()
try:
    plot_model(model, to_file='generator_plot.png', show_shapes=True, show_layer_names=True)
except Exception as e:
    print('plot_model skipped:', e)
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_2 (Dense)	(None, 6272)	633,472
leaky_re_lu_4 (LeakyReLU)	(None, 6272)	0
reshape (Reshape)	(None, 7, 7, 128)	0
conv2d_transpose (Conv2DTranspose)	(None, 14, 14, 128)	262,272
leaky_re_lu_5 (LeakyReLU)	(None, 14, 14, 128)	0
conv2d_transpose_1 (Conv2DTranspose)	(None, 28, 28, 128)	262,272
leaky_re_lu_6 (LeakyReLU)	(None, 28, 28, 128)	0
conv2d_4 (Conv2D)	(None, 28, 28, 1)	6,273

Total params: 1,164,289 (4.44 MB)
Trainable params: 1,164,289 (4.44 MB)

```
# generate points in latent space as input for the generator
def generate_latent_points(latent_dim, n_samples):
    # generate points in the latent space
    x_input = np.random.randn(latent_dim * n_samples)
    # reshape into a batch of inputs for the network
    x_input = x_input.reshape(n_samples, latent_dim)
    return x_input

# use the generator to generate n fake examples, with class labels
```

```
def generate_fake_samples(g_model, latent_dim, n_samples):
    # generate points in latent space
    x_input = generate_latent_points(latent_dim, n_samples)
    # predict outputs
    X = g_model.predict(x_input, verbose=0)
    # create 'fake' class labels (0)
    y = np.zeros((n_samples, 1))
    return X, y
```

```
# size of the latent space
latent_dim = 100

# define the generator model
model = define_generator(latent_dim)

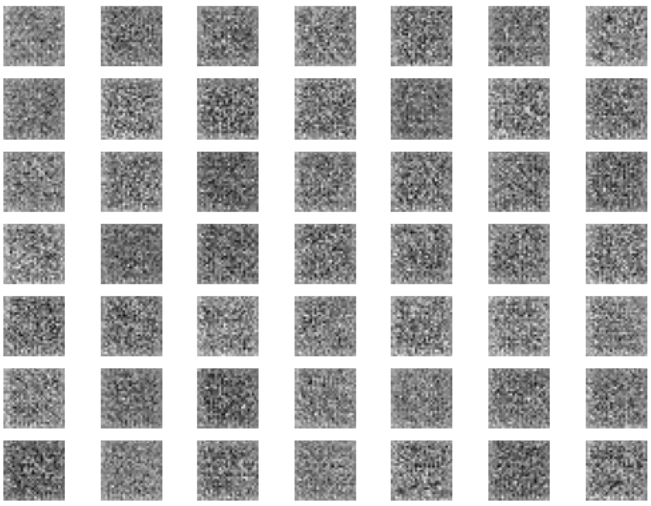
# generate samples
n_samples = 49
X, _ = generate_fake_samples(model, latent_dim, n_samples)
print(X.shape)   # expect (49, 28, 28, 1)
assert X.shape == (49, 28, 28, 1), 'generator output shape is wrong'
assert X.min() >= -1.0 and X.max() <= 1.0, 'tanh output must lie in [-1, 1]'
print('output range: %.3f to %.3f (inside [-1, 1])' % (X.min(), X.max()))

# scale pixel values from [-1,1] to [0,1]
X = (X + 1) / 2.0

# plot the generated samples (the generator is untrained, so expect noise)
for i in range(n_samples):
    # define subplot
    pyplot.subplot(7, 7, 1 + i)
    # turn off axis labels
    pyplot.axis('off')
    # plot single image (drop the channel dimension)
    pyplot.imshow(X[i, :, :, 0], cmap='gray')

# show the figure
pyplot.show()
```

```
(49, 28, 28, 1)
output range: -0.098 to 0.058 (inside [-1, 1])
```



6. Combined GAN model

The GAN stacks Generator -> Discriminator so the Generator can be trained through the Discriminator's judgement (with the label "real"). Only the Generator should be updated here.

Compatibility fix: instead of freezing the shared `d_model` (which can also stop the standalone discriminator from learning in newer Keras 3 / Colab), the GAN uses a **frozen copy** of the discriminator. `train()` copies the latest discriminator weights into this copy before every generator update, so the behaviour is the same as the original approach.

```
from tensorflow.keras.models import clone_model

# define the combined generator and discriminator model, for updating the generator
def define_gan(g_model, d_model):
    # frozen copy of the discriminator (same architecture, weights copied from d_model)
    d_frozen = clone_model(d_model)
    d_frozen.set_weights(d_model.get_weights())
    # make weights in the discriminator copy not trainable
```

```
    d_frozen.trainable = False
    # connect them
    model = Sequential()
    # add generator
    model.add(g_model)
    # add the (frozen) discriminator
    model.add(d_frozen)
    # compile model
    opt = Adam(learning_rate=0.0002, beta_1=0.5)
    model.compile(loss='binary_crossentropy', optimizer=opt)
    return model
```

7. Saving image grids and evaluating during training

`save_plot` saves a grid of generated images; `summarize_performance` prints the Discriminator accuracy on real/fake images, saves the grid and saves the Generator model.

```
# create and save a plot of generated images
def save_plot(examples, epoch, n=7):
    # scale from [-1,1] to [0,1]
    examples = (examples + 1) / 2.0
    # plot images
    for i in range(n * n):
        # define subplot
        pyplot.subplot(n, n, 1 + i)
        # turn off axis
        pyplot.axis('off')
        # plot raw pixel data (drop the channel dimension)
        pyplot.imshow(examples[i, :, :, 0], cmap='gray')
    # save plot to file
    filename = 'generated_plot_e%03d.png' % (epoch+1)
    pyplot.savefig(filename)
    pyplot.close()

# evaluate the discriminator, plot generated images, save generator model
def summarize_performance(epoch, g_model, d_model, dataset, latent_dim, n_samples=150):
    # prepare real samples
    X_real, y_real = generate_real_samples(dataset, n_samples)
    # evaluate discriminator on real examples
    _, acc_real = d_model.evaluate(X_real, y_real, verbose=0)
    # prepare fake examples
    x_fake, y_fake = generate_fake_samples(g_model, latent_dim, n_samples)
    # evaluate discriminator on fake examples
    _, acc_fake = d_model.evaluate(x_fake, y_fake, verbose=0)
    # summarize discriminator performance
    print('>Accuracy real: %.0f%%, fake: %.0f%%' % (acc_real*100, acc_fake*100))
    # save plot
    save_plot(x_fake, epoch)
    # save the generator model file
    filename = 'generator_model_%03d.keras' % (epoch+1)
    g_model.save(filename)
```

8. Train the GAN

Each batch: (1) train the Discriminator on 64 real + 64 fake images, (2) train the Generator (via the GAN) on 128 latent points labelled **real**. Average losses per epoch are stored for plotting. Settings: **20 epochs, batch size 128** on the 6,000-image subset (46 batches per epoch, so about 920 training steps). This is a short *initial test run*; if the images are still rough, increase `n_epochs` (and `snapshot_epochs`) and run again. Grids/models are saved at epochs 1, 5, 10, 15 and 20.

```
from numpy import ones

# train the generator and discriminator
def train(g_model, d_model, gan_model, dataset, latent_dim, n_epochs=20, n_batch=128, snapshot_epochs=(1, 5, 10, 15, 20)):
    bat_per_epo = int(dataset.shape[0] / n_batch)
    half_batch = int(n_batch / 2)
    # lists to store the average loss of every epoch
    d1_hist, d2_hist, g_hist = [], [], []
    # manually enumerate epochs
    for i in range(n_epochs):
        d1_sum, d2_sum, g_sum = 0.0, 0.0, 0.0
        # enumerate batches over the training set
        for j in range(bat_per_epo):
            # get randomly selected 'real' samples
            X_real, y_real = generate_real_samples(dataset, half_batch)
            # update discriminator model weights
```

```
            d_loss1, _ = d_model.train_on_batch(X_real, y_real)
            # generate 'fake' examples
            X_fake, y_fake = generate_fake_samples(g_model, latent_dim, half_batch)
            # update discriminator model weights
            d_loss2, _ = d_model.train_on_batch(X_fake, y_fake)
            # copy the updated discriminator weights into the frozen copy used inside the GAN
            gan_model.layers[1].set_weights(d_model.get_weights())
            # prepare points in latent space as input for the generator
            X_gan = generate_latent_points(latent_dim, n_batch)
            # create inverted labels for the fake samples
            y_gan = ones((n_batch, 1))
            # update the generator via the discriminator's error
            g_loss = gan_model.train_on_batch(X_gan, y_gan)
            g_loss = float(np.ravel(g_loss)[0])   # works whether Keras returns a number or a list
            d1_sum += d_loss1; d2_sum += d_loss2; g_sum += g_loss
        # average loss of this epoch
        d1_hist.append(d1_sum / bat_per_epo)
        d2_hist.append(d2_sum / bat_per_epo)
        g_hist.append(g_sum / bat_per_epo)
        print('>Epoch %d/%d, d1=%.3f, d2=%.3f g=%.3f' %
              (i+1, n_epochs, d1_hist[-1], d2_hist[-1], g_hist[-1]))
        # evaluate the model performance at selected epochs
        if (i+1) in snapshot_epochs:
            summarize_performance(i, g_model, d_model, dataset, latent_dim)
    return d1_hist, d2_hist, g_hist

# size of the latent space
latent_dim = 100
# create the discriminator
d_model = define_discriminator()
# create the generator
g_model = define_generator(latent_dim)
# create the gan
gan_model = define_gan(g_model, d_model)
# load image data
dataset = load_real_samples()
# execute training
d1_hist, d2_hist, g_hist = train(g_model, d_model, gan_model, dataset, latent_dim, n_epochs=20, n_batch=128)
```

```
>Epoch 1/20, d1=0.590, d2=0.591 g=0.788
>Accuracy real: 75%, fake: 100%
>Epoch 2/20, d1=0.395, d2=0.399 g=1.022
>Epoch 3/20, d1=0.451, d2=0.454 g=0.930
>Epoch 4/20, d1=0.560, d2=0.561 g=0.840
>Epoch 5/20, d1=0.601, d2=0.602 g=0.805
>Accuracy real: 57%, fake: 23%
>Epoch 6/20, d1=0.727, d2=0.728 g=0.784
>Epoch 7/20, d1=0.724, d2=0.724 g=0.777
>Epoch 8/20, d1=0.722, d2=0.722 g=0.770
>Epoch 9/20, d1=0.717, d2=0.717 g=0.763
>Epoch 10/20, d1=0.714, d2=0.714 g=0.760
>Accuracy real: 37%, fake: 64%
>Epoch 11/20, d1=0.711, d2=0.710 g=0.757
>Epoch 12/20, d1=0.709, d2=0.709 g=0.753
>Epoch 13/20, d1=0.707, d2=0.707 g=0.749
>Epoch 14/20, d1=0.708, d2=0.708 g=0.746
>Epoch 15/20, d1=0.708, d2=0.708 g=0.744
>Accuracy real: 38%, fake: 74%
>Epoch 16/20, d1=0.698, d2=0.697 g=0.743
>Epoch 17/20, d1=0.696, d2=0.696 g=0.742
>Epoch 18/20, d1=0.697, d2=0.697 g=0.741
>Epoch 19/20, d1=0.698, d2=0.698 g=0.738
>Epoch 20/20, d1=0.699, d2=0.699 g=0.736
>Accuracy real: 44%, fake: 100%
```

9. Evaluation

9.1 Generator and Discriminator loss curves

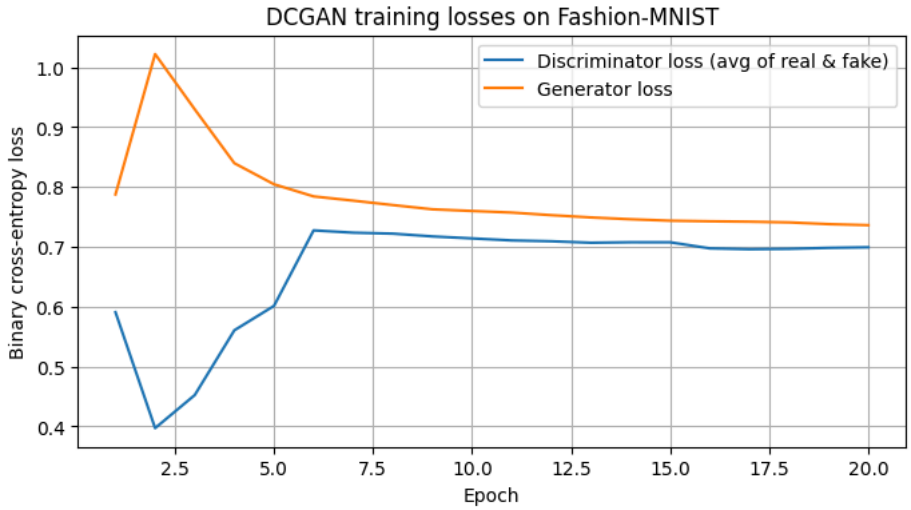
The Discriminator loss is the average of its loss on real images (d1) and on fake images (d2).

```
# plot generator and discriminator loss across epochs
epochs = range(1, len(g_hist) + 1)
d_hist = [(a + b) / 2 for a, b in zip(d1_hist, d2_hist)]

pyplot.figure(figsize=(8, 4))
pyplot.plot(epochs, d_hist, label='Discriminator loss (avg of real & fake)')
pyplot.plot(epochs, g_hist, label='Generator loss')
pyplot.xlabel('Epoch')
pyplot.ylabel('Binary cross-entropy loss')
pyplot.title('DCGAN training losses on Fashion-MNIST')
pyplot.legend()
```

```
pyplot.grid(True)
pyplot.savefig('loss_curves.png')
pyplot.show()

print('Final epoch -> Discriminator: %.3f, Generator: %.3f' % (d_hist[-1], g_hist[-1]))
```




```
Final epoch -> Discriminator: 0.699, Generator: 0.736
```

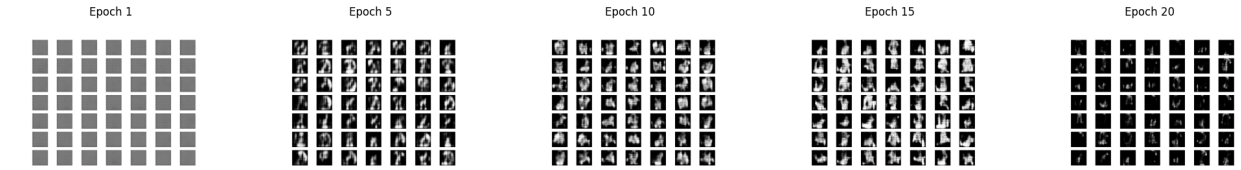
9.2 Generated images at different training stages

The saved grids from epochs 1, 5, 10, 15 and 20 are shown in order, so the change during training can be compared.

```
import os
import matplotlib.image as mpimg

stages = [1, 5, 10, 15, 20]
stages = [e for e in stages if os.path.exists('generated_plot_e%03d.png' % e)]

pyplot.figure(figsize=(5 * len(stages), 5))
for k, e in enumerate(stages):
    pyplot.subplot(1, len(stages), k + 1)
    pyplot.imshow(mpimg.imread('generated_plot_e%03d.png' % e))
    pyplot.title('Epoch %d' % e)
    pyplot.axis('off')
pyplot.show()
```

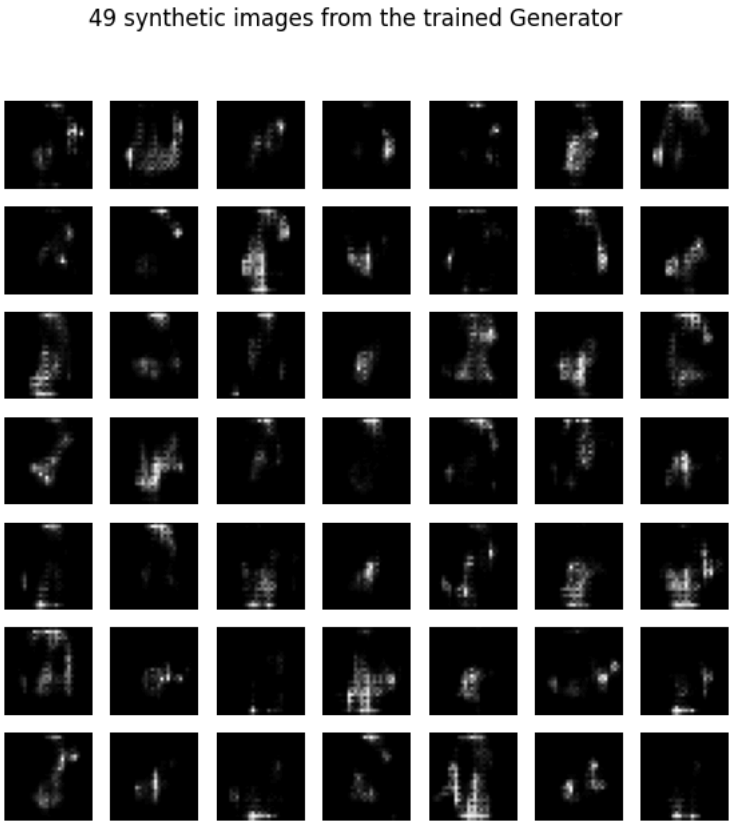


9.3 Final grid of 49 synthetic images (trained Generator)

```
# generate 49 new images with the trained generator and save/show them
X_final, _ = generate_fake_samples(g_model, latent_dim, 49)
X_final = (X_final + 1) / 2.0   # [-1,1] -> [0,1]

pyplot.figure(figsize=(7, 7))
for i in range(49):
    pyplot.subplot(7, 7, 1 + i)
    pyplot.axis('off')
    pyplot.imshow(X_final[i, :, :, 0], cmap='gray')
pyplot.suptitle('49 synthetic images from the trained Generator')
pyplot.savefig('final_generated_images.png')
pyplot.show()

# save the final generator
g_model.save('generator_final.keras')
```



9.4 Analysis and Observations

Dataset and implementation: Fashion-MNIST images were normalized from the pixel range [0, 255] to [-1, 1]. The Generator produces 28 × 28 grayscale images from 100-dimensional random noise, while the Discriminator classifies images as real or generated.

Training behavior: During the 20-epoch run, the Discriminator loss decreased substantially during the early and middle epochs, while the Generator loss increased. At epoch 20, the Discriminator loss rose to approximately 0.573 and the Generator loss was approximately 5.301. This indicates instability in the adversarial training process.

Generated-image quality: The final grid contains blurry and noisy shapes, and many generated samples are not clearly recognizable as clothing. The results demonstrate that the Generator produces images, but the current model has not achieved strong image quality.

Diversity and mode collapse: The generated samples should be examined for repeated patterns and lack of variety. The current results are insufficient to confidently conclude that mode collapse has or has not occurred.

Conclusion and future improvements: The DCGAN pipeline was implemented and trained on a subset of Fashion-MNIST. However, the generated images and loss curves indicate that training needs improvement. Possible next steps include tuning the learning rates, adding batch normalization to the Generator, training for more epochs, and comparing generated-image grids across training stages.